Roteiro 7 — Observabilidade, saúde e resiliência

Disciplina: Desenvolvimento Web com .NET — UNIP. **Fonte:** Roteiros.pdf, páginas 21–23.
Consolidação: 29/10/2026.

Objetivo, introdução e ferramentas

Produzir sinais úteis de diagnóstico e demonstrar controles reproduzíveis de disponibilidade e capacidade. A aplicação usa logging JSON, escopos/correlação, OpenTelemetry, Health Checks, `HttpClient` com resiliência, rate limiting e output caching.

Desenvolvimento e decisões

O middleware registra o ciclo da requisição; o filtro MVC registra o ciclo da ação. São eventos de responsabilidades diferentes, sem copiar o mesmo payload em várias camadas. Os registros excluem corpos sensíveis, senha, cookie e token. O identificador de correlação recebido é limitado a caracteres/tamanho permitidos antes de ser devolvido.

`Telemetria` define a métrica de negócio `portal.tarefas.criadas`, com origem limitada a API/MVC, e o histograma técnico `portal.http.duracao`, com método/status. O método é limitado aos nove verbos conhecidos ou `_OTHER`, evitando cardinalidade arbitrária de verbos personalizados. Não há identificador de usuário, tarefa ou URL individual como dimensão de métrica. A criação inicia o span customizado `tarefa.criar`. OpenTelemetry instrumenta ASP.NET, HttpClient e SQL, com exportação ao console habilitável localmente.

Controle	Configuração e finalidade	Evidência necessária
Liveness	Processo responde; não exige banco	Resposta saudável mesmo em falha da dependência
Readiness	Banco e dependência essencial opcional	200 saudável e 503 na indisponibilidade
Timeout	Limites finitos por tentativa e operação	Chamada controlada encerra no limite
Retry	Duas repetições de falhas transitórias; desabilitado para métodos inseguros	GET recupera; POST não é repetido
Circuit breaker	Janela, limiar e tempo de abertura configurados	Falhas abrem circuito e reduzem novas chamadas
Rate limiting	Política por grupo/cliente; 429 e Retry-After	Carga local finita produz excesso observável
Output cache	Somente painel público, validade de 15 s e tag de invalidação	Reuso antes da escrita e nova resposta depois

Não existe uma dependência HTTP de negócio obrigatória inventada para o portal. `Dependencia:Url` é opcional; quando configurada como essencial, participa do readiness. Os testes controlam essa dependência para reproduzir falhas sem depender de serviço pago ou externo.

O painel público utiliza sempre contexto anônimo, mesmo se a requisição possuir sessão; o cache não recebe estatísticas privadas. Controllers invalidam a tag após escritas. O teste de carga é local e limitado; seus resultados não constituem medição de capacidade produtiva.

Testes, resultados e evidências

`ObservabilidadeTests` verifica limitação, live/ready, retry e circuito. `ApiTests` verifica cache e invalidação. A extensão escolhida é o teste automatizado da política de rate limiting.

Os TRX/logs efetivos estão em STATUS_DE_EXECUCAO.md e `docs/evidencias`. A evidência acadêmica exige também logs correlacionados, ao menos uma métrica e um trace reais de requisição. Código que registra um instrumento não prova exportação; a matriz mantém a pendência até coletar sua saída.

Essa saída foi coletada do processo Development publicado por HTTPS: telemetria-console.txt registra o span `tarefa.criar` com o mesmo TraceId da requisição, logs com correlação, counter `portal.tarefas.criadas` com `origem=api` e valor 1 e histograma `portal.http.duracao` com método/status. O recorte não contém credenciais e comprova exportação real, separadamente do teste com listeners em memória.

Conclusão

Os controles implementados possuem objetivos distintos e cenários reproduzíveis. A aprovação depende dos resultados observados e do material de telemetria efetivamente coletado, com dependências saudáveis e indisponíveis, sem presumir sinais ou desempenho.